

Sistema de Partilha de Ecrã em Direto "Atlas" (Computador + App Android)

Especificação técnica completa, do início ao fim, para uma IA construir de raiz um sistema de suporte remoto **focado exclusivamente na PARTILHA DE ECRÃ**.

Âmbito: cliente partilha o ecrã (PC via navegador e telemóvel via app Android nativa); o técnico vê o ecrã num painel (CRM). SEM controlo remoto / SEM co-browsing.

0. Como usar este documento

Este documento é um **prompt de sistema**. Entregue-o a uma IA de programação full-stack (agente de código) e pede-lhe para *implementar tudo o que está descrito*, respeitando as tecnologias, contratos de API, eventos de sinalização e critérios de aceitação. Tudo o que não estiver aqui pode ser decidido pela IA desde que cumpra os critérios de aceitação da Secção 12.

Âmbito restrito (importante): Este sistema serve apenas para VER o ecrã do cliente. NÃO inclui controlo remoto, toques simulados, gestos, círculo vermelho, nem Serviços de Acessibilidade. Se a IA for tentada a acrescentar controlo remoto, deve **ignorar** — está fora do âmbito.

1. Objetivo e visão do produto

Construir uma aplicação de suporte técnico remoto onde um cliente leigo consegue, com o mínimo de passos, **partilhar o seu ecrã em direto** com um técnico. O técnico acede a um painel (CRM) protegido por login e vê o(s) ecrã(s) dos clientes em tempo real via WebRTC.

- **Cliente em Computador:** abre um link no navegador (Chrome/Edge), clica **COMEÇAR** e partilha o ecrã (API `getDisplayMedia`).
- **Cliente em Android:** instala uma *app nativa* (APK) que captura o ecrã via `MediaProjection` e o transmite por WebRTC. O site deteta Android e mostra um botão grande **INSTALAR APP**.
- **Técnico:** faz login no CRM e vê as sessões ativas (miniaturas ao vivo) e um ecrã ampliado por sessão.

2. Pilha tecnológica (obrigatória)

Camada	Tecnologia	Notas
Backend	Python FastAPI + python-socketio (ASGI)	Porta 8001. Rotas REST sob <code>/api</code> . Socket.io em <code>/api/socketio</code> .
Base de dados	MongoDB (motor async, ex.: Motor)	Coleções <code>sessoes</code> e <code>utilizadores</code> . URL em variável de ambiente <code>MONGO_URL</code> .
	HTML/CSS/JS puro	Servido em <code>frontend/public/</code> . Mobile-first no cliente.

Frontend cliente/ CRM		
Tempo real	Socket.io (sinalização) + WebRTC	STUN Google + (opcional) TURN. 5–15 fps.
App Android	Kotlin , Gradle, WebRTC (getstream), socket.io-client	Captura via MediaProjection . minSdk 24, targetSdk 34.
Autenticação	JWT + bcrypt	Apenas para técnicos. Cliente não faz login.

Todas as URLs, portas e credenciais vêm de variáveis de ambiente. O frontend usa sempre a variável de backend (ex.: **REACT_APP_BACKEND_URL** ou equivalente) para as chamadas. Nunca colocar segredos no código.

3. Personas e fluxos de utilizador

3.1 Cliente (Computador — navegador)

1. Abre **/cliente.html?op=<linkId>**.
2. Vê um cartão com título e um botão grande **COMEÇAR**.
3. Ao clicar, o navegador pede autorização (**getDisplayMedia**); o cliente escolhe o ecrã e confirma.
4. Estado passa a "A partilhar"; o técnico vê o ecrã. Botão *Parar partilha* disponível.

3.2 Cliente (Android — app nativa)

1. Abre o mesmo link no telemóvel. O site **deteta Android** e mostra apenas um botão grande **INSTALAR APP** (esconde o **COMEÇAR** e avisos de partilha web).
2. Instala e abre a app. Toca **COMEÇAR** (um toque).
3. O Android pede autorização de captura (**MediaProjection**) — passo obrigatório do sistema. Confirma.
4. A app transmite o ecrã por WebRTC. Mostra notificação persistente enquanto partilha.

3.3 Técnico (CRM — desktop)

1. Abre **/tecnico.html**, faz login (email + palavra-passe).
2. Vê a lista de sessões do seu operador com **miniaturas ao vivo** das que estão a partilhar.
3. Abre uma sessão para ver o ecrã ampliado. Pode renomear/apagar sessões.

4. Modelo de dados (MongoDB)

Coleção **utilizadores**

```
{
  "email": "tecnico@atlas.pt",      // único
  "password_hash": "<bcrypt>",
  "linkId": "1b460f99db",          // id público usado no link ?op=
  "role": "tecnico"                // ou "admin" (opcional)
}
```

Coleção **sessoes**

```
{
  "id": "<uuid>",                  // id interno da sessão
  "token": "<uuid>",                // token persistente do cliente (reconexão)
  "operador": "tecnico@atlas.pt",  // dono da sessão
  "nome": "",                       // rótulo editável pelo técnico
  "userAgent": "...",
  "inicio": "ISO-8601",
}
```

```

"ultimaAtividade": "ISO-8601",
"online": true,                // socket do cliente ligado
"aPartilhar": false,          // a transmitir ecrã?
"modo": "ecra",
"sid": "<socket-id-do-cliente>" // null quando offline
}

```

Nunca devolver documentos MongoDB em bruto (ObjectId não é serializável). Mapear `_id` → `id` e devolver dicts limpos. Datas em ISO com timezone UTC.

5. API REST (prefixo `/api`)

Método / Rota	Auth	Descrição
<code>POST /api/auth/login</code>	—	Body <code>{email, password}</code> → <code>{token, role}</code> . Verifica bcrypt.
<code>GET /api/me</code>	JWT	Dados do técnico autenticado (email, role, linkId).
<code>GET /api/app-config</code>	—	Público. Devolve <code>{op: <linkId do operador principal>}</code> . Usado pela app Android e pelo link do cliente.
<code>GET /api/ice</code>	—	Devolve <code>{iceServers:[...]}</code> (STUN Google e, opcional, TURN a partir de variáveis de ambiente).
<code>GET /api/sessoes</code>	JWT	Lista as sessões do operador (ordenadas por início desc).
<code>PATCH /api/sessoes/{id}</code>	JWT	Renomear sessão (<code>{nome}</code>).
<code>DELETE /api/sessoes/{id}</code>	JWT	Apagar sessão.

Autorização: `Authorization: Bearer <token>`. Token JWT assinado com segredo em variável de ambiente; expiração razoável (ex.: 12h). Palavras-passe guardadas só como hash bcrypt.

6. Sinalização em tempo real (Socket.io — path `/api/socketio`)

Salas: `op:<email>` (técnico) e `sess:<sessaoId>` (par cliente ↔ técnicos). Estruturas em memória: `sid_operador`, `sid_sessao`, `sessao_tecnicos`.

Evento	Origem → Destino	Descrição
<code>connect</code> (auth.token)	Técnico	Se token válido, entra em <code>op:<email></code> e recebe a lista de sessões.
<code>cliente:hello</code> <code>{op, token?, userAgent}</code>	Cliente → Servidor	Cria/recupera sessão por <code>token</code> . Marca <code>online=true</code> , guarda <code>sid</code> . Responde <code>cliente:sessao {id, token}</code> e atualiza a lista do operador.
<code>cliente:sessao</code> <code>{id, token}</code>	Servidor → Cliente	Confirma a sessão; cliente guarda o token para reconexões.
<code>cliente:partilhar</code> <code>{ativo}</code>	Cliente → Servidor	Define <code>aPartilhar</code> . Se ativo e há técnico na sala, envia <code>tecnico:pronto</code> ao cliente.
<code>sessoes:atualizar</code> <code>[...]</code>	Servidor → Técnico	Lista atualizada das sessões do operador (para o CRM redesenhar).
<code>tecnico:ver</code> <code>{sessaoId}</code>	Técnico → Servidor	

		Entra na sala <code>sess:</code> ; se o cliente estiver online, envia-lhe <code>tecnico:pronto</code> .
<code>tecnico:parar {sessaoId}</code>	Técnico → Servidor	Sai da sala da sessão.
<code>tecnico:pronto {}</code>	Servidor → Cliente	Indica ao cliente que pode criar a oferta WebRTC.
<code>webrtc:offer {sdp}</code>	Cliente → Técnico	Reencaminhado para a sala <code>sess:</code> (com <code>sessaoId</code>).
<code>webrtc:answer {sdp, sessaoId}</code>	Técnico → Cliente	Reencaminhado ao cliente da sessão.
<code>webrtc:ice {candidate, sessaoId?}</code>	Ambos	Troca de candidatos ICE dentro da sala.
<code>disconnect</code>	—	Se era o cliente dono da sessão, marca <code>online=false</code> , <code>aPartilhar=false</code> , <code>sid=null</code> e atualiza o operador.

7. Fluxo WebRTC (papéis e passos)

- **Cliente = offerer** (envia o vídeo do ecrã). **Técnico = answerer** (só recebe — `recvonly`).
- Configuração ICE obtida de `GET /api/ice`. Semântica `unified-plan`, gathering contínuo.
- Sequência: técnico entra na sala (`tecnico:ver`) → servidor envia `tecnico:pronto` ao cliente → cliente cria `offer` e envia `webrtc:offer` → técnico responde `webrtc:answer` → trocam `webrtc:ice` → vídeo flui.
- Fila de ICE: guardar candidatos recebidos antes de `setRemoteDescription` e aplicá-los depois.
- Multi-sessão no CRM: uma `RTCPeerConnection` por sessão (dicionário `pcs[sessaoId]`); miniaturas ligam-se automaticamente às sessões `online && aPartilhar`.

8. Frontend do CLIENTE (`cliente.html` + `cliente.js`)

- Lê `?op=<linkId>`; liga o socket a `/api/socketio` e emite `cliente:hello`. Guarda o `token` em `localStorage`.
- **Deteção de plataforma:**
 - `ehIOS` → mostrar aviso "no iPhone/iPad a partilha web não é suportada".
 - `ehAndroid` (e não em PWA standalone) → esconder COMEÇAR e mostrar botão grande *INSTALAR APP* que descarrega o APK (`/atlas-suporte.apk?v=N`, com parâmetro de versão para evitar cache).
 - Desktop → mostrar *COMEÇAR* que chama `getDisplayMedia({video:true, audio:false})`, tenta baixar `frameRate` para ~10 fps, e inicia a oferta quando `tecnico:pronto`.
- Wake Lock durante a partilha; re-adquirir ao voltar ao separador.
- Botão *Parar partilha*: pára as tracks, fecha a PC, emite `cliente:partilhar {ativo:false}`.
- Design mobile-first, tom verde da marca, botões grandes; cada elemento interativo com `data-testid`.

Cache/atualizações: se usar Service Worker (PWA), servir os estáticos em modo *network-first* e versionar o cache, para que as atualizações apareçam após cada deploy.

9. Frontend do TÉCNICO (`tecnico.html` + `tecnico.js`)

- Ecrã de login (email/palavra-passe) → guarda o JWT, chama `GET /api/me` e `GET /api/sesoes`.
- Liga o socket com `auth:{token}`; recebe `sesoes:atualizar` e redesenha a grelha de cartões.

- Cada cartão: nome/dispositivo, estado (online / a partilhar), miniatura de vídeo ao vivo, ações (Ver, Renomear, Apagar).
- Modal "Ver": vídeo ampliado (`<video autoplay playsinline>`) da sessão selecionada.
- Liga-se automaticamente por WebRTC às sessões `online && aPartilhar` (miniaturas).

10. App ANDROID nativa (Kotlin)

10.1 Permissões / Manifesto

```
INTERNET, ACCESS_NETWORK_STATE, WAKE_LOCK,
FOREGROUND_SERVICE, FOREGROUND_SERVICE_MEDIA_PROJECTION, POST_NOTIFICATIONS
<service android:foregroundServiceType="mediaProjection" />
```

10.2 Componentes

- **MainActivity**: ecrã simples com botão *COMEÇAR*. Ao tocar: obtém o operador via `GET /api/app-config`, pede permissão de notificações (Android 13+) e depois `MediaProjectionManager.createScreenCaptureIntent()`. Com o resultado, arranca o serviço em primeiro plano.
- **ScreenShareService** (foreground, tipo `mediaProjection`): mantém a notificação e cria a sessão WebRTC.
- **Sessão WebRTC**: `PeerConnectionFactory` + `ScreenCapturerAndroid(projectionData)` a ~15 fps na resolução do ecrã; liga o socket (`path=/api/socketio`, transporte websocket), emite `cliente:hello {op, token?}`, ao receber `cliente:sessao` guarda o token e emite `cliente:partilhar {ativo:true}`; ao receber `tecnico:pronto` cria a `offer` e envia `webrtc:offer`; aplica `webrtc:answer` e troca `webrtc:ice`.

10.3 Bibliotecas Gradle

```
implementation 'io.getstream:stream-webrtc-android:1.3.8'
implementation 'io.socket:socket.io-client:2.1.0'
// + androidx core/appcompat/material, viewBinding
```

10.4 Assinatura / distribuição

- Gerar uma **keystore de release** (`keytool`) e assinar o APK (`assembleRelease`, esquema v2). Evita o aviso de "app de debug/não confiável" do Play Protect.
- Servir o APK como ficheiro estático (ex.: `/atlas-suporte.apk`) para o botão INSTALAR APP.
- Incrementar `versionCode` / `versionName` a cada nova versão (para instalar como atualização) e versionar o link (`?v=N`) para contornar cache.

Nota Android: o pedido de autorização de captura (`MediaProjection`) é obrigatório do sistema e não pode ser suprimido. O objetivo é que o utilizador o confirme com um único toque.

11. Autenticação e segurança

- Login verifica bcrypt; devolve JWT assinado (segredo em variável de ambiente).
- Semeadura (seeding) idempotente das contas de técnico a partir de configuração; nunca gravar palavras-passe em claro.
- Dependência FastAPI `utilizador_atual` valida o Bearer token em todas as rotas protegidas.
- CORS configurado para a origem do frontend. Socket.io e `/api` sempre pela rede (nunca em cache).

12. Critérios de aceitação (Definition of Done)

1. Um técnico faz login e vê a lista das suas sessões.

2. Num **computador**, o cliente abre o link, clica COMEÇAR, autoriza e o técnico **vê o ecrã em direto** (miniatura + ampliado) com latência baixa.
3. Num **Android**, o link mostra só INSTALAR APP; após instalar e tocar COMEÇAR (+ autorização do sistema), o técnico **vê o ecrã do telemóvel**.
4. Ao parar/fechar, a sessão fica **offline** e a miniatura deixa de transmitir.
5. O APK está **assinado em release** e instala sem o aviso de "app de debug".
6. Nenhuma funcionalidade de controlo remoto está presente (apenas visualização).

13. Sugestão de testes

- **Backend (curl):** login → token; **GET /api/sesoes** com Bearer; **GET /api/app-config** e **GET /api/ice** devolvem JSON válido.
- **Cliente Web (Playwright):** com UA Android → botão INSTALAR APP visível e COMEÇAR escondido; com UA desktop → o inverso.
- **WebRTC:** abrir cliente (desktop) + CRM em dois contextos e confirmar que o vídeo chega ao técnico.
- **Android:** instalar o APK assinado, ligar e confirmar a partilha no CRM (teste em dispositivo real).

Este prompt descreve um sistema de **partilha de ecrã** (visualização) para Computador (navegador) e Android (app nativa). Mantém o âmbito restrito: sem controlo remoto, sem gestos, sem co-browsing. Segue os contratos de API e eventos de sinalização exatamente como especificado para garantir compatibilidade entre cliente, app e CRM.